```
iex> GenServer.call(pid, {:add, 5, 10})
15

iex> GenServer.cast(pid, :announce)
:ok
[info]  Genny: The last result I computed was 15
```

Let's break each function down to understand what is happening. The start_link/2 function on line 5 starts the GenServer and kicks off initialization. start_link is able to pass initialization arguments into the process. We're using this to accept a speaker parameter that's placed in the GenServer state.

The init/1 function on line 9 receives the provided arguments and starts with a given state. Maps or structs are great for state because they are easy to update and access in the message handlers.

GenServer.cast/2 sends a message to the process. The handle_cast/2 callback on line 13 is then invoked with the message contents and the state of the process. Cast messages are asynchronous, which means that the calling process doesn't receive a reply and won't wait for the message to be processed.

GenServer.call/2 sends a message to the process. The handle_call/3 callback on line 21 is invoked with the message contents, some info about the calling process, and the state of the process. Call messages are synchronous and will wait for a default 5 seconds before raising an error. Call messages receive a response, which allows you to extract information from the GenServer.

The init, handle_cast, and handle_call functions are all executed inside of the GenServer process. start_link, GenServer.cast, and GenServer.call are all performed inside of the calling process.

Expand Our GenServer

It's rare to use GenServer functions directly outside of a GenServer-implemented module. It's a best practice to create wrapper functions that invoke them for you. Put these helper functions above start_link/2:

```
def announce(server \\ __MODULE__) do
  GenServer.cast(server, :announce)
end

def add(a, b, server \\ __MODULE__) do
  GenServer.call(server, {:add, a, b})
end
```

This code does the exact same thing as calling the GenServer functions manually, but callers no longer need to concern themselves with the internals of GenServer.